

Analysis Plan

Regan Yang, Sahil Dadhwal, Carlos Anguiano, Ritesh Patro

1. Candidate IP set size after each phase

What we measure: The number of /24 IP prefixes remaining after Phase 1 (GeoLite2 query), Phase 2 (reverse DNS filtering), and Phase 3 (WHOIS and FCC NBM filtering).

How we measure it: Our Python script logs the count after each phase for every school.

Comparison: Compare set sizes between locally hosted vs cloud hosted schools, and between urban/well-funded vs rural/underfunded schools. Smaller final sets mean higher confidence.

2. Identification success confidence metric

What we measure: Whether we successfully identified a school's IP, based on how many sources agree on the best candidate IP.

How we measure it: For each school's top candidate IP, count how many sources agree it belongs to the school (reverse DNS hostname match, WHOIS organization match, FCC NBM provider match). If 2 or more agree, count it as a successful identification. If 0 or 1, count it as a failure.

Comparison: Compare the success confidence metric for locally hosted schools vs cloud hosted schools. We expect locally hosted to have a much higher rate.

3. Hosting type (local vs cloud)

What we measure: Whether each school's website resolves to a local IP near the school or to a cloud provider like AWS, Cloudflare, Aptegy/Fastly.

How we measure it: Run nslookup or ping on the school's website domain, then check the resulting IP against known cloud provider IP ranges (or use a service like IPinfo to check hosting classification).

Comparison: Compare the proportion of cloud hosted vs locally hosted schools in urban/well-funded vs rural/underfunded categories. Test our hypothesis that rural/underfunded schools are more likely to be cloud hosted.

4. Network provider serving each school

What we measure: Which ISP or organization owns the IPs in the final candidate set for each school.

How we measure it: WHOIS lookup on the final candidate IPs. Record the organization name and ASN.

Comparison: Compare provider diversity between urban and rural schools. Check if rural schools tend to have fewer provider options (one ISP) vs urban schools (multiple ISPs).

Sample data and code (for measure #1 and #2):

```
import csv

# Sample data we would collect
sample_data = [
    {"school": "Davis High", "type": "urban", "hosting": "local",
```

```

    "phase1_count": 1200, "phase2_count": 45, "phase3_count": 8,
    "rdns_match": True, "whois_match": True, "nbm_match": True},
{"school": "Johns Elementary", "type": "rural", "hosting": "cloud",
 "phase1_count": 800, "phase2_count": 3, "phase3_count": 0,
 "rdns_match": False, "whois_match": False, "nbm_match": False},
{"school": "Oak Tree Middle", "type": "urban", "hosting": "cloud",
 "phase1_count": 1500, "phase2_count": 5, "phase3_count": 2,
 "rdns_match": False, "whois_match": False, "nbm_match": True},
{"school": "Napa High", "type": "rural", "hosting": "local",
 "phase1_count": 600, "phase2_count": 30, "phase3_count": 5,
 "rdns_match": True, "whois_match": False, "nbm_match": True},
]

# Count how many of the sources will agree for each of the schools
for s in sample_data:
    print(f"School: {s['school']}")
    s["confidence"] = sum([s["rdns_match"],
                           s["whois_match"],
                           s["nbm_match"]])

    s["identified"] = s["confidence"] >= 2

# Analysis 1: average candidate set size by hosting type
local = [s for s in sample_data if s["hosting"] == "local"]
cloud = [s for s in sample_data if s["hosting"] == "cloud"]

print("Avg final candidates (local):",
      sum(s["phase3_count"] for s in local) / len(local))
print("Avg final candidates (cloud):",
      sum(s["phase3_count"] for s in cloud) / len(cloud))

# Analysis 2: success rate by hosting type
local_success = sum(1 for s in local if s["identified"]) / len(local)
cloud_success = sum(1 for s in cloud if s["identified"]) / len(cloud)

print(f"Success rate (local): {local_success}")
print(f"Success rate (cloud): {cloud_success}")

# Analysis 3: compare urban vs rural
urban = [s for s in sample_data if s["type"] == "urban"]
rural = [s for s in sample_data if s["type"] == "rural"]

print("Avg final candidates (urban):",
      sum(s["phase3_count"] for s in urban) / len(urban))
print("Avg final candidates (rural):",
      sum(s["phase3_count"] for s in rural) / len(rural))

```

Expected output from sample data:

Avg final candidates (local): 6.5
 Avg final candidates (cloud): 1.0
 Success rate (local): 100%
 Success rate (cloud): 0%
 Avg final candidates (urban): 5.0
 Avg final candidates (rural): 2.5